

Task1_QA_Report_BiswamberBehera

Automation & QA Developer Take-Home Skills Assessment

Candidate Name: Biswamber Behera

Role: Automation & QA Developer

Task: Task 1 — Web App QA & Debug Report

Tested Application: Conduit / RealWorld Demo Web App

Application URL: <https://react-redux.realworld.io>

Testing Type: Manual Exploratory QA

Browser/Environment: Google Chrome, Windows Desktop

Testing Date: 21 May 2026

1. Application Selected

For this QA task, I tested the **Conduit / RealWorld Demo Web App**, which is a Medium-style blogging platform. The app allows users to sign up, log in, create articles, edit articles, delete articles, view feeds, and log out.

The initially suggested public demo URL showed a **403 Forbidden** error during access, so I selected an alternate RealWorld implementation that supports the same core user flows required in the assignment.

2. Scope of Testing

The purpose of this testing was to check whether the application is ready for production use from a functional, usability, and QA perspective.

I tested the following main user flows:

1. User sign-up
2. User login
3. Create new article
4. Edit existing article
5. Delete article
6. Logout
7. Form validation
8. Error handling
9. UI feedback
10. Basic accessibility and layout behavior

The focus was not only on the happy path, but also on edge cases such as invalid input, slow responses, unclear messages, long content, and destructive actions.

3. Bug Report Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Unclear error message on invalid login	1. Open the Sign In page. 2. Enter an unregistered email or wrong password. 3. Click Sign In.	Expected: The app should show a clear message such as “Invalid email or password.” Actual: The error message is generic or unclear and does not properly guide the user.	Medium	Backend/API error is not mapped properly to a user-friendly frontend message.
2	Weak validation on sign-up form	1. Open the Sign Up page. 2. Submit the form with empty fields. 3. Try invalid email format. 4. Try a very short password.	Expected: The app should show inline validation before submission for username, email, and password. Actual: Validation mostly appears after submission, and the feedback is not always specific enough.	High	Client-side validation rules are incomplete or the frontend depends too much on backend validation.
3	No clear loading state after submitting forms	1. Open Sign In, Sign Up, or New Article page. 2. Submit the form on a slow network. 3. Observe the submit button and page behavior.	Expected: The submit button should be disabled and show a loading state such as “Signing in...” or “Publishing...”. Actual: The user may not know whether the request is processing and can click the button multiple times.	Medium	Async request state is not properly connected to the UI button state.
4	Long article title or body affects readability	1. Login to the app. 2. Create a new article. 3. Add a very long title without spaces or a very long body paragraph. 4. Publish and view the article/feed.	Expected: Long content should wrap, truncate, or be handled gracefully without disturbing the layout. Actual: Long content can reduce readability and may disturb spacing/layout.	Medium	CSS rules for word wrapping, max-width, overflow, and content constraints are insufficient.
5	Delete article action has weak confirmation and feedback	1. Login and create an article. 2. Open the article page. 3. Click Delete Article.	Expected: The app should show a confirmation prompt before deleting and display success/failure feedback after deletion. Actual: The delete action feels abrupt, and the user may not receive enough confirmation.	High	Destructive action does not include a confirmation step or strong notification handling.
6	Accessibility issue: form fields and focus states need improvement	1. Navigate through the app using the keyboard Tab key. 2. Move through sign-in, sign-up, and article forms. 3. Observe focus visibility and field guidance.	Expected: All interactive elements should have strong visible focus states, accessible labels, and clear keyboard navigation. Actual: Focus visibility and field guidance are limited in some areas.	Medium	UI relies heavily on placeholder text and default styling instead of stronger accessibility patterns.

4. Root-Cause Analysis

Selected Issue: No clear loading state after submitting forms.

When a user submits a form such as **Sign In**, **Sign Up**, or **New Article**, the application sends an asynchronous request to the backend API. During this waiting period, the interface does not clearly communicate that the request is in progress. Because the submit button remains visually similar and may not be disabled, users can become unsure whether their click worked. This can lead to repeated clicks, duplicate requests, and confusion, especially on slow networks.

The likely root cause is that the frontend does not maintain or expose a proper loading state for each form submission. The form component should track when an API request starts and ends. A state variable such as `isLoading` or `isSubmitting` can be used to disable the submit button and update the button text. For example, the button could show “Signing in...” during login or “Publishing...” during article creation. The loading state should reset after either a successful response or an error response. Error handling should also be connected to the same request lifecycle so the user receives a clear message if the API fails. This would make the application feel more reliable and prevent accidental duplicate actions.

5. Testing Notes and Assumptions

- The application was tested as a production-like public demo web app.
- Some behavior may vary depending on network speed, browser, backend availability, and demo database reset behavior.
- The issues listed are based on manual exploratory QA and user-facing production readiness.
- Since source code access was not used, suspected causes are based on observed behavior and common frontend/backend patterns.
- If source code access were available, the next step would be to inspect form components, API service functions, error handling utilities, and CSS rules.

6. Recommended Priority Order

1. Improve sign-up and login validation/error messages.
2. Add loading and disabled states to all submit buttons.
3. Add confirmation and clear feedback for destructive actions such as delete.
4. Improve layout handling for long user-generated content.
5. Improve keyboard accessibility, visible focus states, and form labels.

7. Conclusion

The Conduit / RealWorld demo app supports the required core flows such as sign-up, login, article creation, editing, deletion, and logout. However, several issues reduce its production readiness. The biggest risks are unclear validation, weak feedback during asynchronous actions, and insufficient handling of destructive actions such as article deletion.

These issues are fixable with better frontend state management, stronger validation, clearer error mapping, and improved UI/accessibility standards. Overall, the app works for basic flows, but it needs improvements before being considered reliable for production users.